

SQL區數統計語法

2024年6月25日

ⓘ 此對話可能會透露建立者的個人化資料，這些資料並不會與他人共享，並且可能大幅影響模型的回應方式。

如果要對一個SQL資料表的某一數值欄進行數值區數出現的次數統計，要怎麼下SQL語法，以500為每個統計區間大小。

要對 SQL 資料表中的某一數值欄進行以 500 為區間大小的區數出現次數統計，可以使用 `CASE` 語句來創建區間，然後使用 `GROUP BY` 和 `COUNT` 函數進行統計。假設資料表名為 `table\_name`，數值欄位名為 `value\_column`，以下是範例 SQL 語法：

```
sql 複製程式碼

SELECT
  CASE
    WHEN value_column >= 0 AND value_column < 500 THEN '0-499'
    WHEN value_column >= 500 AND value_column < 1000 THEN '500-999'
    WHEN value_column >= 1000 AND value_column < 1500 THEN '1000-1499'
    WHEN value_column >= 1500 AND value_column < 2000 THEN '1500-1999'
    -- 可以根據需要繼續添加更多區間
    ELSE '2000+'
  END AS value_range,
  COUNT(*) AS count
FROM
  table_name
GROUP BY
  value_range
ORDER BY
  value_range;
```

這個 SQL 語法會根據 `value\_column` 的數值範圍分組，並統計每個範圍內數值出現的次數。你可以根據你的實際數據範圍和需求來調整 `CASE` 語句中的區間範圍。

改以對數值的方式定義統計區間

要以數值的方式定義統計區間，可以利用數學運算來計算每個數值所在的區間。以下是一種方法，可以利用整數除法來確定區間，假設每個區間的大小為 500：

```
sql 複製程式碼

SELECT
  CONCAT(FLOOR(value_column / 500) * 500, '-', FLOOR(value_column / 500) * 500 + 499) AS value_range,
  COUNT(*) AS count
FROM
  table_name
GROUP BY
  FLOOR(value_column / 500)
ORDER BY
  FLOOR(value_column / 500);
```

這段 SQL 語法的工作原理如下：

- 使用 `FLOOR(value\_column / 500)` 計算每個數值所在的區間索引。
- 使用 `CONCAT` 和 `FLOOR` 函數生成區間字符串表示，例如 "0-499", "500-999" 等。
- 使用 `GROUP BY` 和 `COUNT` 函數對每個區間進行統計。

4. 最後使用 `ORDER BY` 來按區間排序。

這樣，能夠靈活地對每個區間大小為 500 的數值範圍進行統計。

欄位名為Loaded Samples Count，資料表名為sampleCount_Main



根據您提供的資料表名和欄位名，以下是修改後的 SQL 語法：

```
sql 複製程式碼

SELECT
    CONCAT(FLOOR([Loaded Samples Count] / 500) * 500, '-', FLOOR([Loaded Samples Count] / 500)) AS count
    COUNT(*) AS count
FROM
    sampleCount_Main
GROUP BY
    FLOOR([Loaded Samples Count] / 500)
ORDER BY
    FLOOR([Loaded Samples Count] / 500);
```

這段 SQL 語法會根據 `Loaded Samples Count` 欄位的數值範圍分組，並統計每個範圍內數值出現的次數。請注意，根據不同的 SQL 資料庫，可能需要調整語法以符合其具體的 SQL 標準。例如，有些資料庫可能需要將欄位名稱中的空格用引號括起來或使用不同的引號形式。

出現[11:40:27] Error while executing SQL query on database
'dataset_total_labels_count': no such function: CONCAT

請以SQLite支援的語法撰寫。



SQLite 不支持 `CONCAT` 函數，但你可以使用 `||` 運算符來連接字符串。以下是針對 SQLite 語法的修改版本：

```
sql 複製程式碼

SELECT
    (FLOOR([Loaded Samples Count] / 500) * 500) || '-' || (FLOOR([Loaded Samples Count] / 500)) AS count
    COUNT(*) AS count
FROM
    sampleCount_Main
GROUP BY
    FLOOR([Loaded Samples Count] / 500)
ORDER BY
    FLOOR([Loaded Samples Count] / 500);
```

這段 SQL 語法利用 `||` 運算符來連接區間範圍的字符串表示，並且依舊使用 `FLOOR` 函數來計算每個數值所在的區間索引。這樣可以在 SQLite 中實現相同的效果。

用對數函數的方式定義統計區間



要使用對數函數來定義統計區間，可以通過對數運算來計算每個數值的區間。假設我們以500為基數來劃分區間，可以使用 `LOG` 函數來實現。SQLite使用 `LOG10` 函數計算對數，並且你可以使用指數運算符來計算指數。

下面是針對SQLite語法的SQL查詢語句：

```
sql 複製程式碼

SELECT
    CAST(POWER(10, FLOOR(LOG10([Loaded Samples Count] / 500.0))) * 500 AS INT) || '-' || CAST(
    COUNT(*) AS count
FROM
```

```

sampleCount_Main
GROUP BY
    FLOOR(LOG10([Loaded Samples Count] / 500.0))
ORDER BY
    FLOOR(LOG10([Loaded Samples Count] / 500.0));

```

這段SQL語法的工作原理如下：

1. 使用`LOG10([Loaded Samples Count] / 500.0)`計算`Loaded Samples Count`除以500的對數值。
2. 使用`FLOOR`將對數值向下取整，得到區間索引。
3. 使用`POWER`函數將區間索引轉換回指數形式，並乘以500來確定區間的開始值。
4. 使用`CAST`將計算結果轉換為整數。
5. 使用`||`運算符將開始值和結束值連接起來，形成區間字符串。
6. 使用`GROUP BY`和`COUNT`函數對每個區間進行統計。
7. 使用`ORDER BY`對區間進行排序。

這樣可以在SQLite中根據對數函數來劃分區間並進行統計。

對數區數的意思是指統計區間像0~1、1~10、10~100、100~1000這樣。



明白了，您想要的是基於對數範圍的統計區間，比如 0~1, 1~10, 10~100, 100~1000 等。這可以使用對數函數來劃分這些區間。

以下是基於這些對數範圍的 SQL 語法，假設每個區間為 10 的倍數：

```

sql
複製程式碼

SELECT
  CASE
    WHEN [Loaded Samples Count] >= 0 AND [Loaded Samples Count] < 10 THEN '0-9'
    WHEN [Loaded Samples Count] >= 10 AND [Loaded Samples Count] < 100 THEN '10-99'
    WHEN [Loaded Samples Count] >= 100 AND [Loaded Samples Count] < 1000 THEN '100-999'
    WHEN [Loaded Samples Count] >= 1000 AND [Loaded Samples Count] < 10000 THEN '1000-9999'
    -- 可以根據需要繼續添加更多區間
    ELSE '10000+'
  END AS value_range,
  COUNT(*) AS count
FROM
  sampleCount_Main
GROUP BY
  CASE
    WHEN [Loaded Samples Count] >= 0 AND [Loaded Samples Count] < 10 THEN '0-9'
    WHEN [Loaded Samples Count] >= 10 AND [Loaded Samples Count] < 100 THEN '10-99'
    WHEN [Loaded Samples Count] >= 100 AND [Loaded Samples Count] < 1000 THEN '100-999'
    WHEN [Loaded Samples Count] >= 1000 AND [Loaded Samples Count] < 10000 THEN '1000-9999'
    ELSE '10000+'
  END
ORDER BY
  MIN([Loaded Samples Count]);

```

這段 SQL 語法會根據`Loaded Samples Count`的值劃分到不同的對數區間，並計算每個區間內數值出現的次數。每個`CASE`語句定義了一個範圍，並將相應的值分組和統計。

用函數的方式來自動產生合適的統計區間。



要用函數的方式來自動產生合適的統計區間，尤其是基於對數範圍的統計區間（例如：0~1、1~10、10~100、100~1000），可以利用對數和指數運算來生成這些區間。SQLite 中可以使用`LOG10`和`POWER`函數來實現。

以下是基於這些對數範圍的 SQL 語法：

```
sql 複製程式碼

SELECT
  CASE
    WHEN [Loaded Samples Count] < 1 THEN '0-1'
    ELSE CAST(POWER(10, FLOOR(LOG10([Loaded Samples Count]))) AS TEXT) || '-' ||
          CAST(POWER(10, FLOOR(LOG10([Loaded Samples Count])) + 1) - 1 AS TEXT)
    END AS value_range,
    COUNT(*) AS count
FROM
  sampleCount_Main
GROUP BY
  CASE
    WHEN [Loaded Samples Count] < 1 THEN '0-1'
    ELSE CAST(POWER(10, FLOOR(LOG10([Loaded Samples Count]))) AS TEXT) || '-' ||
          CAST(POWER(10, FLOOR(LOG10([Loaded Samples Count])) + 1) - 1 AS TEXT)
    END
ORDER BY
  MIN([Loaded Samples Count]);
```

這段 SQL 語法的工作原理如下：

1. 使用 `LOG10([Loaded Samples Count])` 計算 `Loaded Samples Count` 的常用對數。
2. 使用 `FLOOR(LOG10([Loaded Samples Count]))` 來確定對數範圍的下限。
3. 使用 `POWER(10, FLOOR(LOG10([Loaded Samples Count])))` 來計算下限的數值。
4. 使用 `POWER(10, FLOOR(LOG10([Loaded Samples Count])) + 1) - 1` 來計算上限的數值。
5. 將下限和上限轉換為字符串並用 `||` 運算符連接，形成區間字符串表示。
6. 使用 `GROUP BY` 和 `COUNT` 函數對每個區間進行統計。
7. 使用 `ORDER BY MIN([Loaded Samples Count])` 來按區間的最小值排序。

這樣可以自動生成基於對數範圍的統計區間，並統計每個區間內數值出現的次數。

統計區間的頭尾為整數，不要有小數點，並新增三欄顯示該統計範圍內，最多的前三個類別值，類別值的資料欄位名是index



為了實現這一需求，可以使用以下步驟：

1. 使用對數範圍生成區間，並確保區間的頭尾為整數。
2. 統計每個區間內的值。
3. 新增三個欄位，顯示每個區間內最多的前三個類別值（按 `index` 欄位統計）。

下面是實現此需求的 SQL 語法，假設資料表名為 `sampleCount\_Main`，類別值欄位名為 `index`：

```
sql 複製程式碼

WITH Ranges AS (
  SELECT
    [Loaded Samples Count],
    index,
    CASE
      WHEN [Loaded Samples Count] < 1 THEN '0-1'
      ELSE CAST(POWER(10, FLOOR(LOG10([Loaded Samples Count]))) AS TEXT) || '-' ||
            CAST(POWER(10, FLOOR(LOG10([Loaded Samples Count])) + 1) - 1 AS TEXT)
      END AS value_range
  FROM sampleCount_Main
),
Counts AS (
  SELECT
    value_range,
    COUNT(*) AS count,
    index
```

```

FROM Ranges
GROUP BY value_range, index
),
TopCategories AS (
SELECT
    value_range,
    index,
    ROW_NUMBER() OVER (PARTITION BY value_range ORDER BY count DESC) AS rank
FROM Counts
),
TopCategoriesFiltered AS (
SELECT
    value_range,
    MAX(CASE WHEN rank = 1 THEN index ELSE NULL END) AS top1,
    MAX(CASE WHEN rank = 2 THEN index ELSE NULL END) AS top2,
    MAX(CASE WHEN rank = 3 THEN index ELSE NULL END) AS top3
FROM TopCategories
WHERE rank <= 3
GROUP BY value_range
)
SELECT
    R.value_range,
    COUNT(*) AS count,
    TC.top1,
    TC.top2,
    TC.top3
FROM
    Ranges R
LEFT JOIN
    TopCategoriesFiltered TC
ON R.value_range = TC.value_range
GROUP BY
    R.value_range
ORDER BY
    MIN([Loaded Samples Count]);

```

這段 SQL 語法的工作原理如下：

1. Ranges CTE：根據 `Loaded Samples Count` 計算區間範圍。
2. Counts CTE：按區間範圍和 `index` 分組並計算每個 `index` 出現的次數。
3. TopCategories CTE：使用 `ROW\_NUMBER()` 函數為每個區間內的 `index` 排序，按出現次數降序排列。
4. TopCategoriesFiltered CTE：過濾出每個區間內出現次數最多的前三個 `index`。
5. 最後選擇區間範圍、統計數量和前三個 `index`，並按區間範圍排序。

這樣的語法可以滿足您的需求，生成對數範圍並顯示每個範圍內最多的前三個類別值。

出現[11:46:25] Error while executing SQL query on database 'dataset_total_labels_count': near "index": syntax error錯誤



在 SQLite 中，`index` 是保留字，所以我們需要對它進行引用。在 SQLite 中，可以使用雙引號 (``) 來引用保留字。以下是修正後的 SQL 語法：

```

sql 複製程式碼

WITH Ranges AS (
SELECT
    [Loaded Samples Count],
    "index",
    CASE
        WHEN [Loaded Samples Count] < 1 THEN '0-1'
        ELSE CAST(POWER(10, FLOOR(LOG10([Loaded Samples Count])))) AS TEXT) || '-' ||
            CAST(POWER(10, FLOOR(LOG10([Loaded Samples Count])) + 1) - 1 AS TEXT)
    END AS value_range
FROM sampleCount_Main

```

```

),
Counts AS (
  SELECT
    value_range,
    COUNT(*) AS count,
    "index"
  FROM Ranges
  GROUP BY value_range, "index"
),
TopCategories AS (
  SELECT
    value_range,
    "index",
    ROW_NUMBER() OVER (PARTITION BY value_range ORDER BY count DESC) AS rank
  FROM Counts
),
TopCategoriesFiltered AS (
  SELECT
    value_range,
    MAX(CASE WHEN rank = 1 THEN "index" ELSE NULL END) AS top1,
    MAX(CASE WHEN rank = 2 THEN "index" ELSE NULL END) AS top2,
    MAX(CASE WHEN rank = 3 THEN "index" ELSE NULL END) AS top3
  FROM TopCategories
  WHERE rank <= 3
  GROUP BY value_range
)
SELECT
  R.value_range,
  COUNT(*) AS count,
  TC.top1,
  TC.top2,
  TC.top3
FROM
  Ranges R
LEFT JOIN
  TopCategoriesFiltered TC
ON R.value_range = TC.value_range
GROUP BY
  R.value_range
ORDER BY
  MIN([Loaded Samples Count]);

```

這樣修正後應該能夠在 SQLite 中正常執行。請注意每次使用 `index` 時，都用雙引號將其引起來。

統計區間不要有顯示小數位數，取整數顯示。



要確保統計區間不顯示小數位數並顯示整數，可以對範圍的計算結果進行四捨五入或截取整數部分。以下是修改後的 SQL 語法，保證統計區間的顯示為整數：

```

sql 複製程式碼

WITH Ranges AS (
  SELECT
    [Loaded Samples Count],
    "index",
    CASE
      WHEN [Loaded Samples Count] < 1 THEN '0-1'
      ELSE CAST(CAST(POWER(10, FLOOR(LOG10([Loaded Samples Count])))) AS INT) AS TEXT) || '
      CAST(CAST(POWER(10, FLOOR(LOG10([Loaded Samples Count])) + 1) - 1 AS INT) AS TEXT)
    END AS value_range
  FROM sampleCount_Main
),
Counts AS (
  SELECT
    value_range,
    COUNT(*) AS count,
    "index"

```

```

FROM Ranges
GROUP BY value_range, "index"
),
TopCategories AS (
SELECT
    value_range,
    "index",
    ROW_NUMBER() OVER (PARTITION BY value_range ORDER BY count DESC) AS rank
FROM Counts
),
TopCategoriesFiltered AS (
SELECT
    value_range,
    MAX(CASE WHEN rank = 1 THEN "index" ELSE NULL END) AS top1,
    MAX(CASE WHEN rank = 2 THEN "index" ELSE NULL END) AS top2,
    MAX(CASE WHEN rank = 3 THEN "index" ELSE NULL END) AS top3
FROM TopCategories
WHERE rank <= 3
GROUP BY value_range
)
SELECT
    R.value_range,
    COUNT(*) AS count,
    TC.top1,
    TC.top2,
    TC.top3
FROM
    Ranges R
LEFT JOIN
    TopCategoriesFiltered TC
ON R.value_range = TC.value_range
GROUP BY
    R.value_range
ORDER BY
    MIN([Loaded Samples Count]);

```

這段 SQL 語法對於對數範圍的計算結果使用 `CAST` 函數轉換為整數類型，從而保證區間範圍的顯示為整數而不是小數。這樣，統計區間的頭尾都將顯示為整數。

各統計區間顯示的最多類別數量，增加至前十多的類別，並且在類別數量後面加上(該類別的對應Loaded Samples Count欄位數值)。



要顯示每個統計區間內最多的前十個類別，並在類別數量後面加上對應的 `Loaded Samples Count` 數值，可以通過以下 SQL 語法實現：

```

sql 複製程式碼

WITH Ranges AS (
SELECT
    [Loaded Samples Count],
    "index",
    CASE
        WHEN [Loaded Samples Count] < 1 THEN '0-1'
        ELSE CAST(CAST(POWER(10, FLOOR(LOG10([Loaded Samples Count])))) AS INT) AS TEXT) || '
        CAST(CAST(POWER(10, FLOOR(LOG10([Loaded Samples Count])) + 1) - 1 AS INT) AS TE
    END AS value_range
FROM sampleCount_Main
),
Counts AS (
SELECT
    value_range,
    "index",
    [Loaded Samples Count],
    COUNT(*) AS count
FROM Ranges
GROUP BY value_range, "index", [Loaded Samples Count]
)

```

```

//
TopCategories AS (
    SELECT
        value_range,
        "index",
        [Loaded Samples Count],
        ROW_NUMBER() OVER (PARTITION BY value_range ORDER BY count DESC) AS rank
    FROM Counts
),
TopCategoriesFiltered AS (
    SELECT
        value_range,
        MAX(CASE WHEN rank = 1 THEN "index" || ' (' || [Loaded Samples Count] || ')' ELSE NULL
        MAX(CASE WHEN rank = 2 THEN "index" || ' (' || [Loaded Samples Count] || ')' ELSE NULL
        MAX(CASE WHEN rank = 3 THEN "index" || ' (' || [Loaded Samples Count] || ')' ELSE NULL
        MAX(CASE WHEN rank = 4 THEN "index" || ' (' || [Loaded Samples Count] || ')' ELSE NULL
        MAX(CASE WHEN rank = 5 THEN "index" || ' (' || [Loaded Samples Count] || ')' ELSE NULL
        MAX(CASE WHEN rank = 6 THEN "index" || ' (' || [Loaded Samples Count] || ')' ELSE NULL
        MAX(CASE WHEN rank = 7 THEN "index" || ' (' || [Loaded Samples Count] || ')' ELSE NULL
        MAX(CASE WHEN rank = 8 THEN "index" || ' (' || [Loaded Samples Count] || ')' ELSE NULL
        MAX(CASE WHEN rank = 9 THEN "index" || ' (' || [Loaded Samples Count] || ')' ELSE NULL
        MAX(CASE WHEN rank = 10 THEN "index" || ' (' || [Loaded Samples Count] || ')' ELSE NUL
    FROM TopCategories
    WHERE rank <= 10
    GROUP BY value_range
)
SELECT
    R.value_range,
    COUNT(*) AS count,
    TC.top1,
    TC.top2,
    TC.top3,
    TC.top4,
    TC.top5,
    TC.top6,
    TC.top7,
    TC.top8,
    TC.top9,
    TC.top10
FROM
    Ranges R
LEFT JOIN
    TopCategoriesFiltered TC
ON R.value_range = TC.value_range
GROUP BY
    R.value_range
ORDER BY
    MIN([Loaded Samples Count]);

```

這段 SQL 語法執行以下步驟：

1. Ranges CTE：根據 `Loaded Samples Count` 計算區間範圍。
2. Counts CTE：按區間範圍、`index` 和 `Loaded Samples Count` 分組並計算每個 `index` 出現的次數。
3. TopCategories CTE：使用 `ROW\_NUMBER()` 函數為每個區間內的 `index` 排序，按出現次數降序排列。
4. TopCategoriesFiltered CTE：過濾出每個區間內出現次數最多的前十個 `index`，並將 `Loaded Samples Count` 添加到每個 `index` 後。
5. 最後選擇區間範圍、統計數量和前十個 `index`，並按區間範圍排序。

Topk 欄位類別的選擇要依 Loaded Samples Count 的數值由高到低排序。



要依據 `Loaded Samples Count` 的數值由高到低排序並選出每個統計區間內最多的前十個類別，並在

每個類別後面加上對應的 `Loaded Samples Count` 數值，可以使用以下 SQL 語法：

```
sql 複製程式碼

WITH Ranges AS (
    SELECT
        [Loaded Samples Count],
        "index",
        CASE
            WHEN [Loaded Samples Count] < 1 THEN '0-1'
            ELSE CAST(CAST(POWER(10, FLOOR(LOG10([Loaded Samples Count]))) AS INT) AS TEXT) || '
                CAST(CAST(POWER(10, FLOOR(LOG10([Loaded Samples Count])) + 1) - 1 AS INT) AS TEXT)
        END AS value_range
    FROM sampleCount_Main
),
Counts AS (
    SELECT
        value_range,
        "index",
        [Loaded Samples Count],
        COUNT(*) AS count
    FROM Ranges
    GROUP BY value_range, "index", [Loaded Samples Count]
),
TopCategories AS (
    SELECT
        value_range,
        "index",
        [Loaded Samples Count],
        ROW_NUMBER() OVER (PARTITION BY value_range ORDER BY [Loaded Samples Count] DESC, count)
    FROM Counts
),
TopCategoriesFiltered AS (
    SELECT
        value_range,
        MAX(CASE WHEN rank = 1 THEN "index" || ' (' || [Loaded Samples Count] || ')' ELSE NULL) AS top1,
        MAX(CASE WHEN rank = 2 THEN "index" || ' (' || [Loaded Samples Count] || ')' ELSE NULL) AS top2,
        MAX(CASE WHEN rank = 3 THEN "index" || ' (' || [Loaded Samples Count] || ')' ELSE NULL) AS top3,
        MAX(CASE WHEN rank = 4 THEN "index" || ' (' || [Loaded Samples Count] || ')' ELSE NULL) AS top4,
        MAX(CASE WHEN rank = 5 THEN "index" || ' (' || [Loaded Samples Count] || ')' ELSE NULL) AS top5,
        MAX(CASE WHEN rank = 6 THEN "index" || ' (' || [Loaded Samples Count] || ')' ELSE NULL) AS top6,
        MAX(CASE WHEN rank = 7 THEN "index" || ' (' || [Loaded Samples Count] || ')' ELSE NULL) AS top7,
        MAX(CASE WHEN rank = 8 THEN "index" || ' (' || [Loaded Samples Count] || ')' ELSE NULL) AS top8,
        MAX(CASE WHEN rank = 9 THEN "index" || ' (' || [Loaded Samples Count] || ')' ELSE NULL) AS top9,
        MAX(CASE WHEN rank = 10 THEN "index" || ' (' || [Loaded Samples Count] || ')' ELSE NULL) AS top10
    FROM TopCategories
    WHERE rank <= 10
    GROUP BY value_range
)
SELECT
    R.value_range,
    COUNT(*) AS count,
    TC.top1,
    TC.top2,
    TC.top3,
    TC.top4,
    TC.top5,
    TC.top6,
    TC.top7,
    TC.top8,
    TC.top9,
    TC.top10
FROM
    Ranges R
LEFT JOIN
    TopCategoriesFiltered TC
ON R.value_range = TC.value_range
GROUP BY
    R.value_range
ORDER BY
    R.value_range
```

```
MIN([Loaded Samples Count]));
```

這段 SQL 語法的工作原理如下：

1. **Ranges CTE**：根據 `Loaded Samples Count` 計算區間範圍。
2. **Counts CTE**：按區間範圍、`index` 和 `Loaded Samples Count` 分組並計算每個 `index` 出現的次數。
3. **TopCategories CTE**：使用 `ROW_NUMBER()` 函數為每個區間內的 `index` 排序，按 `Loaded Samples Count` 數值由高到低排序，若 `Loaded Samples Count` 相同，則按出現次數降序排列。
4. **TopCategoriesFiltered CTE**：過濾出每個區間內出現次數最多的前十個 `index`，並將 `Loaded Samples Count` 添加到每個 `index` 後。
5. 最後選擇區間範圍、統計數量和前十個 `index`，並按區間範圍排序。